

Collaborative Threshold Watermarking

Tameem Bakr^{*1} Anish Ambreth^{*1} Nils Lukas¹

Abstract

In federated learning (FL), K clients jointly train a model without sharing raw data. Because each participant invests data and compute, clients need mechanisms to later prove the provenance of a jointly trained model. Model watermarking embeds a hidden signal in the weights, but naive approaches either do not scale with many clients as per-client watermarks dilute as K grows, or give any individual client the ability to verify and potentially remove the watermark. We introduce (t, K) -threshold watermarking: clients collaboratively embed a shared watermark during training, while only coalitions of at least t clients can reconstruct the watermark key and verify a suspect model. We secret-share the watermark key τ so that coalitions of fewer than t clients cannot reconstruct it, and verification can be performed without revealing τ in the clear. We instantiate our protocol in the white-box setting and evaluate it on image classification tasks on both IID and non-IID partitions, as well as language models fine-tuning setting. Our watermark remains detectable at scale ($K = 128$) with minimal accuracy loss and stays above the detection threshold ($z \geq 4$) under attacks including adaptive fine-tuning using up to 20% of the training data. Code is available at <https://github.com/tameemalaa/collaborative-threshold-watermark>.

1. Introduction

Federated learning (FL) enables multiple parties to train machine learning models collaboratively without sharing raw data. FL has already been deployed at scale, for example, in Google Keyboard (Beaufays et al., 2019; Yang et al., 2018), Apple’s voice recognition (Granqvist et al., 2020), and other privacy-sensitive applications (NVIDIA,

2021; Oldenhof et al., 2023; Muzellec et al., 2024). Recent work by Sani et al. (2024); Ye et al. (2024) envisions training large language models (LLMs) from scratch via FL to democratize model training and ownership.

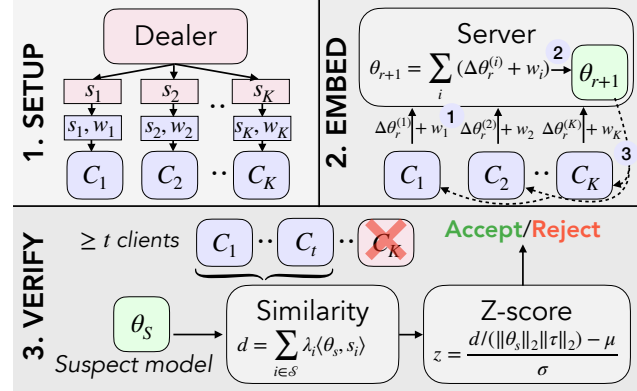


Figure 1. An overview of collaborative threshold watermarking. SETUP is a one-time procedure to distribute Shamir shares s_i to all clients from which they derive additive shares w_i . EMBED modifies the FL algorithm to embed our watermark, VERIFY allows any coalition of $\geq t$ clients to compute the watermark test statistic.

Training large models consumes vast compute and data resources. However, once a model is jointly trained in FL, any client could redistribute it without the consent of any other client. Clients must trust each other *not* to leak the model before engaging in the FL protocol. However, FL often operates in settings with limited trust (e.g., without contractual obligations), so *trustless*, distributed mechanisms are needed to verify a model’s provenance.

Model watermarking (Uchida et al., 2017; Adi et al., 2018) is a solution to verify a model’s provenance. Watermarking embeds a hidden signal that can later be detected by accessing the model using a secret watermarking key. Watermarking methods for FL must (i) preserve the model’s utility, (ii) be reliably detectable, and (iii) be robust against unauthorized removal attempts (Boenisch, 2021; Lukas et al., 2021). Naive approaches fail at scale: if each client embeds its own watermark, the signal each client can embed into the model diminishes as K grows. If all clients own the watermark key, then any individual client can verify and potentially remove the watermark, undermining robustness. A protocol is needed where K clients embed collaboratively, but verification requires a coalition of at least $t \leq K$ clients.

¹Department of Machine Learning, MBZUAI, Abu Dhabi, UAE. Correspondence to: Tameem Bakr <tameem.bakr@mbzuai.ac.ae>.

Threshold verification is useful in settings where ownership is collective: for example, in a healthcare FL consortium of five hospitals, choosing $t = 3$ prevents any single hospital from unilaterally claiming provenance or removing the watermark, while still enabling majority-based verification.

We propose the first method for (t, K) -threshold model watermarking. Our protocol combines (i) secret sharing, so that any coalition of at least t clients can reconstruct a watermark key τ for verification, and (ii) secure aggregation, so that clients can embed shares w_k that sum to τ during training with untrusted servers. Secure aggregation hides individual client updates, but the server still observes the aggregated global model trajectory. Our goal is to prevent any party from reconstructing τ unless they hold at least t shares. Verification is performed in the white-box setting, computing the test statistic directly from shares without reconstructing the secret key τ in the clear, and computing a calibrated, one-sided z -score on a suspect model. Across CIFAR-10, CIFAR-100 (Krizhevsky, 2009), and Tiny ImageNet (Le & Yang, 2015), we show that our approach scales to many clients (we test up to $K = 128$), when a baseline watermark diminishes as the number of clients grows and falls below the detection threshold once $K \geq 16$. Additional experiments on CIFAR-100 under balanced Dirichlet label skew shows that watermark detectability remains stable under non-IID statistical heterogeneity, and a GPT-2 Small (Radford et al., 2019) experiment on WikiText-2 (Merity et al., 2017) indicates that the method also extends beyond image classification. Our watermark has a small impact on model utility and is robust to a wide range of both adaptive and non-adaptive removal attacks when access to training data is limited. Our watermark has a small impact on model accuracy and is robust to a wide range of both adaptive and non-adaptive removal attacks when access to training data is limited.

1.1. Contributions.

Our contributions can be summarized as follows:

- We propose the first (t, K) -threshold watermark for FL that scales to many clients and ensures only $\geq t$ clients can collectively verify the presence of a watermark.
- We empirically show that our watermark has a negligible impact on the model’s accuracy and is reliably detectable via a one-sided, calibrated z -test.
- Our watermark remains detectable under pruning up to 90%, 4-bit quantization, and adaptive fine-tuning with up to 20% of the training data.

2. Background

We describe federated learning (FL), secure aggregation and model watermarking. Then we review cryptographic

primitives such as commitments and secret sharing.

2.1. Federated Learning (FL)

FL enables multiple clients to collaboratively train a shared model without exchanging raw data (Kairouz et al., 2021). Each client i has a dataset of size n_i , and the global model θ is trained to minimize the overall weighted loss

$$F(\theta) = \frac{1}{n} \sum_{i=1}^K n_i F_i(\theta), \quad (1)$$

where $F_i(\theta)$ is the loss on client i ’s local data and $n = \sum_{i=1}^K n_i$. The server maintains a global model θ_r at round r . Training proceeds across many rounds: (1) The server sends the current global model to clients, (2) clients update it on their local data, and (3) the server aggregates these updates and sends the updated model back to the clients.

Secure Aggregation Secure aggregation is a key primitive in FL for protecting client privacy. While FL avoids sharing raw data, individual client updates can still leak sensitive information if observed by the server. Secure aggregation mitigates this risk by ensuring that the server learns only an aggregate over client updates, not any individual contribution. We write $y \leftarrow \text{SECAGG}(\{x_k\}_{k=1}^K)$ for a secure aggregation protocol that outputs $y = \sum_{k=1}^K x_k$ to the server, while keeping each individual x_k hidden from the server. We treat this as a standard FL building block and refer to known constructions such as that of Bonawitz et al. (2017). Such protocols incur communication overhead linear in the number of clients, modest additional local computation, and are robust to client dropouts up to a fixed threshold. Since our protocol invokes secure aggregation twice only over model-sized vectors, compared to once in normal FL training, its communication and computation costs are asymptotically equivalent to those of FL training.

2.2. Model Watermarking

A model watermark (Uchida et al., 2017) is a hidden signal that can be extracted from the model using a secret watermarking key. Formally, any watermarking method is defined by the following three algorithms:

- $\tau \leftarrow \text{SETUP}()$: Samples a secret key and prepares the information needed for embedding and verification.
- $\theta_w \leftarrow \text{EMBED}(\theta, \tau)$: Given a model’s parameters θ and a secret key τ , return watermarked parameters.
- $y \leftarrow \text{VERIFY}(\theta_s, \tau)$: Detect the watermark in a *suspect* model θ_s using the key τ , and output a z -score which rejects the null hypothesis that the detected signal is present due to random chance.

A *white-box* watermark requires that VERIFY has access to

the model’s parameters for verification, whereas a black-box watermark requires only API access. By definition, black-box schemes are also verifiable in the white-box setting.

2.3. Commitment Schemes

A *commitment scheme* (Brassard et al., 1988; Katz & Lindell, 2020) is a two-phase protocol between a sender and receiver. It consists of a COMMIT algorithm to fix a value m with randomness r , producing a commitment $\mathcal{C}$, and an OPEN algorithm where the sender reveals (m, r) for verification. It has two characteristics: (i) **Hiding**: Commitments reveal nothing about m . (ii) **Binding**: It is infeasible to open the same commitment to two different values. We write $\mathcal{C} \leftarrow \text{COMMIT}(\tau; \rho)$ to denote that we publish a public nonce ρ and a commitment $\mathcal{C}$ to the watermark key τ .

2.4. Secret Sharing

A (t, n) -threshold secret sharing scheme (Stinson, 2005; Katz & Lindell, 2020) splits a secret s into shares $s_1, \dots, s_n$ such that any $|\mathcal{S}| \geq t$ shares reconstruct s , while the joint distribution of any $|\mathcal{S}| < t$ shares is independent of s .

Shamir’s threshold scheme. In Shamir’s classical (t, n) -threshold scheme (Shamir, 1979), the dealer samples a random degree- $(t - 1)$ polynomial $P(x)$ over a finite field $\mathbb{F}_q$ with $P(0) = s$, where s is the secret. Each participant i is assigned a distinct nonzero $x_i \in \mathbb{F}_q$ and receives the share (x_i, s_i) where $s_i = P(x_i)$. Any set of t shares uniquely determines $P(x)$, while any set of fewer than t shares reveals no information about s . We denote share generation by $\{s_i\}_{i=1}^n \leftarrow \text{SHAMIRSHARE}(s, t, n)$ (with fixed public evaluation points, e.g., $x_i = i$), and reconstruction by $s \leftarrow \text{SHAMIRRECONSTRUCT}(\{s_i\}_{i \in \mathcal{S}})$ for any $|\mathcal{S}| \geq t$.

Additive (embedding) shares. We call vectors $w_1, \dots, w_K$ *additive shares* of a secret τ if $\sum_{i=1}^K w_i = \tau$. We highlight that if all clients hold Shamir shares of a secret τ , they can cooperate to derive embedding shares via public Lagrange coefficients, while retaining Shamir’s (t, n) -threshold secrecy for verification. Specifically, for fixed public evaluation points $\{x_i\}_{i=1}^K$, the Lagrange coefficient

$$\lambda_i = \prod_{j \neq i} \frac{0 - x_j}{x_i - x_j}, \quad \text{such that } \tau = \sum_{i=1}^K \lambda_i s_i. \quad (2)$$

is a publicly computable constant that depends only on the evaluation points. Using (2), each client locally computes its embedding share as $w_i = \lambda_i s_i$.

3. Threat Model

We consider a standard FL setup: K clients want to train a joint model without revealing their raw data, and each client contributes (i) data and (ii) computation. Clients want the ability to collectively verify the presence of a watermark, for example, to retain ownership or to deter unauthorized use of the model.

Trusted Dealer. We consider both settings where (i) a trusted dealer is available (which we focus on in the main paper), and (ii) where a trusted dealer is unavailable. In the latter case, clients need to invoke a dealer-free distributed key generation algorithm described in the Appendix.

Clients and Server. Each client holds private local datasets, and the client set is fixed across rounds. All clients are *honest-but-curious*, meaning they follow the protocol during training, but may form coalitions of $< t$ clients to remove the watermark post-training. We assume that clients do not trust the server, e.g., since a client could act as the server. The goal is that the server does not learn the secret shares and that τ cannot be reconstructed without at least t client shares. Each client’s objective is to support collaborative verification and resist watermark removal by any coalition of fewer than t clients while maintaining utility.

Adversary. We consider a white-box adversary with access to the entire training trajectory, i.e., all intermediate model checkpoints. The adversary knows the secret shares of any coalition of $< t$ clients, and they have auxiliary data not used during FL, which they can use for further fine-tuning. Unless stated otherwise, we assume the attacker’s auxiliary data is labeled and drawn from the same distribution as the FL training data. We focus on post-training watermark-removal attacks, and malicious training-time adversaries are out of scope for our work. The adversary’s objective is to obtain a model (i) with high accuracy¹ (ii) that does not contain the watermark.

Design Goals. Our watermarking method should (i) allow shared ownership, where only a coalition of $\geq t$ clients can verify the watermark in a model. We want the watermark to (ii) scale with an arbitrary number of clients K , and (iii) operate in the presence of an untrusted server that must not learn the secret watermark key τ . (iv) The watermark must be robust against removal attacks after FL training, and (v) have a low false positive rate on models trained without knowledge of our watermarking method.

¹Our adversary lacks sufficient data to train a high-utility model from scratch. Otherwise, there is no need to engage in FL.

Algorithm 1 Trusted dealer setup (One-time operation).

Require: K clients, threshold t , public evaluation points $\{x_k\}_{k=1}^K$

- 1: Dealer samples watermark key τ and public nonce ρ
- 2: Publish $(\rho, \mathcal{C} \leftarrow \text{COMMIT}(\tau; \rho))$
- 3: $\{s_k\}_{k=1}^K \leftarrow \text{SHAMIRSHARE}(\tau, t, K)$ (client shares)
- 4: **for** each client k **do**
- 5: Send s_k to k over an authenticated, private channel
- 6: $\lambda_k \leftarrow \prod_{j=1, j \neq k}^K \frac{0 - x_j}{x_k - x_j}$ {Lagrange Coefficient}
- 7: $w_k \leftarrow \lambda_k s_k$ (so $\sum_{k=1}^K w_k = \tau$)
- 8: **end for**
- 9: Dealer deletes τ

4. Conceptual Approach

The key idea is to use Shamir secret sharing so that only coalitions of size $\geq t$ can reconstruct τ , while individual clients can still embed *shares* of τ during training under secure aggregation. We instantiate this idea as three components, mirroring the watermarking primitives of Section 2.2: a one-time **SETUP** that distributes Shamir shares of τ to the K clients, an **EMBED** procedure that integrates these shares into the FedAvg update under secure aggregation, and a reconstruction-free **VERIFY** step that evaluates the watermark test directly from shares without ever materializing τ in the clear.

4.1. Watermark Setup

Trusted Dealer. Algorithm 1 implements our **SETUP** procedure that runs once at the beginning of the FL process among all K clients. The goal is to distribute verification and embedding shares of the secret key τ to all clients. We sample the watermark key as $\tau \sim \mathcal{N}(0, I_d)$.

Because the aggregation server is untrusted with τ , it never receives τ in the clear. Instead, after the setup, each client k holds a Shamir share s_k of τ and derives an *embedding share* w_k using the (public) Lagrange coefficient λ_k for evaluation at 0 over the full client set:

$$w_k \leftarrow \lambda_k \cdot s_k \quad \text{so that} \quad \sum_{k=1}^K w_k = \tau. \quad (3)$$

For fixed public evaluation points (e.g., $x_k = k$), the coefficients $\{\lambda_k\}_{k=1}^K$ are fixed constants and can be precomputed once. This lets clients embed shares locally under **SECAGG** (Section 2.1) while the server learns only the aggregated watermark contribution.

Dealer-free setup. If a trusted dealer is unavailable, clients can run a dealer-free distributed key generation (DKG) protocol to obtain Shamir shares of an implicit random τ , revealing it to any party (Appendix B). At a high

level, each client k samples a degree- $(t-1)$ polynomial P_k with random constant term and privately sends evaluations $P_k(x_i)$ to each client i , who sums the received values to obtain its share. This requires $O(K^2)$ authenticated point-to-point messages (each carrying one fixed-point vector share in $\mathbb{Z}_q^d$) and $O(K^2 d)$ total communication.²

4.2. Watermark Embedding

At each round r , client k performs local training to obtain $\theta_r^{(k)}$ and computes the local model update

$$\Delta\theta_r^{(k)} := \theta_r^{(k)} - \theta_{r-1}.$$

A naive embedding strategy would add a fixed-strength watermark perturbation in every round. However, client update magnitudes can vary substantially across rounds and clients due to data heterogeneity, optimizer dynamics, and learning-rate schedules. Fixed-strength embedding can therefore be unstable, either degrading model utility or becoming undetectable when update norms fluctuate.

To stabilize the embedding strength, we adaptively scale the watermark magnitude based on recent update norms. Each client maintains an exponential moving average (EMA) of its update magnitudes. Let $\text{ema}_k^{(r)}$ denote the EMA tracker for client k at round r , updated with decay parameter β as

$$\text{ema}_k^{(r)} = \beta \cdot \text{ema}_k^{(r-1)} + (1 - \beta) \cdot \|\Delta\theta_r^{(k)}\|_2. \quad (4)$$

Each client then calibrates the strength of its watermark perturbation as

$$\text{scale}_k = c \cdot \|\Delta\theta_r^{(k)}\|_2 \cdot \text{ema}_k^{(r)}, \quad (5)$$

where c is a global watermark strength hyperparameter.

To preserve the additive-share structure of the watermark and prevent the server from learning individual scaling factors, clients submit scale_k via secure aggregation. The server obtains only the global scaling factor

$$\text{scale}_{\text{total}} \leftarrow \text{SECAGG}(\{\text{scale}_k\}_{k=1}^K) = \sum_{k=1}^K \text{scale}_k, \quad (6)$$

which is broadcast to all clients. Using a single global $\text{scale}_{\text{total}}$ is necessary to ensure that the sum of scaled shares holds to a scaled τ ; per-client scaling would break this additive structure. This procedure adds one secure aggregation step per round, incurring modest communication overhead.

Per-client embedding. Clients keep τ secret-shared and derive embedding shares w_k from their Shamir shares s_k as in Eq. 3. After local training, clients contribute

²Appendix B.2 reports empirical per-client runtime and communication/computation overhead for the dealer-free DKG.

scale_k to SECAGG (Section 2.1) so the server learns only scale_{total} = $\sum_k \text{scale}_k$, which it broadcasts. Each client forms a watermarked model $u_k \leftarrow \theta_r^{(k)} + \text{scale}_{\text{total}} w_k$ and contributes u_k to SECAGG. The server learns only the aggregate and sets:

$$\theta_r = \frac{1}{K} \text{SECAGG}\left(\{u_k\}_{k=1}^K\right) \quad (7)$$

Because $\sum_k w_k = \tau$, the magnitude of the watermark drift per round is $(\text{scale}_{\text{total}}/K) \tau$.

Client subsampling and dropouts. If only a subset of the clients $S_r \subseteq \{1, \dots, K\}$ participates in round r , the same idea applies as long as $|S_r| \geq t$, which is that each participating client $k \in S_r$ computes the round-specific Lagrange coefficient $\lambda_k^{(S_r)}$ for evaluation at 0 over points $\{x_i\}_{i \in S_r}$ and sets $w_k^{(S_r)} \leftarrow \lambda_k^{(S_r)} s_k$. Then $\sum_{k \in S_r} w_k^{(S_r)} = \tau$, and the aggregate update under SECAGG yields the same watermark direction (with K replaced by $|S_r|$ in the averaging). Under full participation, $\lambda_k^{(S_r)} = \lambda_k$ is fixed and can be precomputed once. If $|S_r| < t$, clients skip watermark embedding for that round.

Weighted FedAvg. If the server applies weighted aggregation, i.e.,

$$\theta_r = \sum_{k \in S_r} a_{k,r} (\theta_r^{(k)} + \text{scale}_{\text{total},r} w_k), \quad (8)$$

with public weights $a_{k,r} > 0$ such that $\sum_{k \in S_r} a_{k,r} = 1$, each participating client $k \in S_r$ rescales its embedding share to $\tilde{w}_{k,r} \leftarrow \frac{1}{a_{k,r}} w_k$. Then, as shown in Eq. (8), we obtain

$$\sum_{k \in S_r} a_{k,r} \tilde{w}_{k,r} = \sum_{k \in S_r} a_{k,r} \frac{1}{a_{k,r}} w_k = \sum_{k \in S_r} w_k. \quad (9)$$

So the aggregated update preserves the same watermark direction as in the unweighted case (cf. Eq. (9)). As before, if $|S_r| < t$, clients skip watermark embedding for that round.

4.3. White-box Watermark Verification

The simplest procedure to verify the watermark reconstructs the key τ from Shamir shares $\{s_i\}_{i \in \mathcal{S}}$ via polynomial interpolation, but then the secret is leaked in the clear. However, reconstructing the key is not required since our test depends only on the inner product $\langle \theta_s, \tau \rangle$. A coalition can compute it directly from shares without ever materializing τ . For a coalition $\mathcal{S}$ with $|\mathcal{S}| \geq t$ and Lagrange coefficients $\{\lambda_i\}_{i \in \mathcal{S}}$ for evaluation at 0, we have $\tau = \sum_{i \in \mathcal{S}} \lambda_i s_i$, hence

$$\langle \theta_s, \tau \rangle = \sum_{i \in \mathcal{S}} \lambda_i \langle \theta_s, s_i \rangle. \quad (10)$$

Each client i locally computes the scalar $\langle \theta_s, s_i \rangle$ and the coalition sums the weighted scalars via secure aggregation to obtain $\langle \theta_s, \tau \rangle$. Together with $\|\theta_s\|_2$ and a fixed $\|\tau\|_2$ implied by the key distribution, this suffices to compute $\cos(\theta_s, \tau)$ and the final z -score.

Given a suspect model θ_s , we compute the cosine similarity between θ_s and τ and calculate the z -score

$$z = \frac{\cos(\theta_s, \tau) - \mu}{\sigma} \quad (11)$$

where μ and σ denote the mean and standard deviation of cosine similarities between unwatermarked models and random vectors, which we empirically validate is approximately normal (see Figure 2). We consider a model θ_s watermarked if $z \geq 4$, which corresponds to a false positive rate of $\approx 3.2 \cdot 10^{-5}$.

Our method. Algorithm 2 implements our collaborative threshold watermarking protocol for FL. In the setup phase, a trusted dealer or a DKG protocol assigns each of the K clients a Shamir share s_k of a secret watermark τ , from which clients precompute public Lagrange coefficients λ_k and embedding shares w_k . In each training round r , the server broadcasts the current global model θ_{r-1} , clients perform local training to obtain updates $\Delta\theta_r^{(k)}$ and track their magnitudes, and a secure aggregation step computes a global scaling factor $\text{scale}_{\text{total},r}$ that adaptively determines the watermark strength. Each client embeds its watermark share into its local model update by submitting $u_r^{(k)}$ via secure aggregation, allowing the server to average the embedded updates into a new global model θ_r . After training completes, any coalition $S \subseteq \{1, \dots, K\}$ with $|S| \geq t$ can jointly verify the watermark on a suspect model θ_s by computing a verification statistic $z(\theta_s)$ from their shares optionally checking consistency with a public commitment and the watermark is accepted if $z(\theta_s) \geq z^*$.

Note that $\text{LOCALTRAIN}(\theta, \mathcal{D}_k)$ denotes standard local training on client k 's dataset. Any coalition of fewer than t clients cannot reconstruct τ (by Shamir secrecy), and the only value revealed during verification is the final statistic z . Moreover, with reconstruction-free verification, the same watermark key can be checked repeatedly across many checkpoints or models without revealing τ in the clear.

5. Experiments

Experimental Setup. Experiments were conducted on the CIFAR-10, CIFAR-100 datasets (Krizhevsky, 2009) and Tiny ImageNet (Le & Yang, 2015). For image classification experiments, we use the ResNet-18 architecture (He et al., 2016); all models were randomly initialized before training, and unless otherwise stated, results are averaged across three random seeds. We use FedAvg and vary the number of

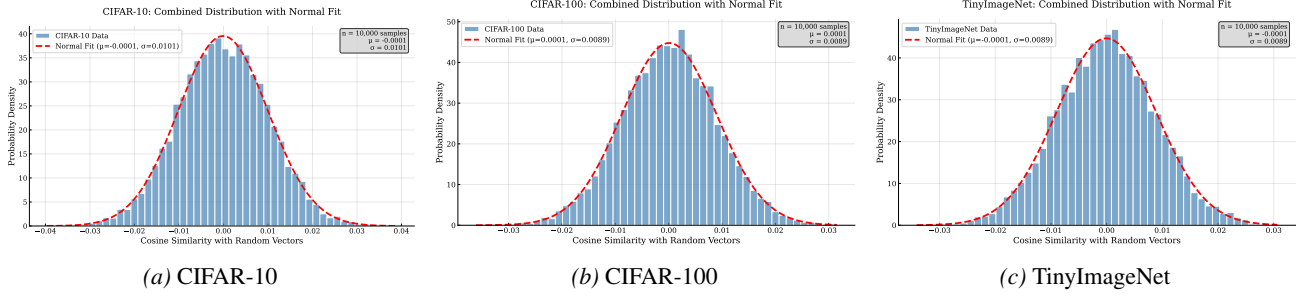


Figure 2. Combined cosine similarity distributions for different datasets on ResNet18 models, with fitted normal distributions.

Algorithm 2 Threshold watermarking protocol.

Require: K clients, threshold t , rounds T , strength c , EMA decay β , public evaluation points $\{x_k\}_{k=1}^K$

- 1: **Setup:** Run SETUPTRUSTEDDEALER (Alg. 1) or dealer-free SETUPDKG (Appendix B.1) to distribute Shamir shares $\{s_k\}_{k=1}^K$ of τ and publish nonce ρ (and optional commitment $\mathcal{C}$).
- 2: **Precompute:** Each client computes its public Lagrange coefficient λ_k and embedding share $w_k \leftarrow \lambda_k s_k$
- 3: **for** round $r = 1$ to T **do**
- 4: Server broadcasts θ_{r-1}
- 5: **for** each client k in parallel **do**
- 6: $\theta_r^{(k)} \leftarrow \text{LOCALTRAIN}(\theta_{r-1}, \mathcal{D}_k)$
- 7: $\Delta_k \leftarrow \theta_r^{(k)} - \theta_{r-1}$
- 8: $\text{ema}_k \leftarrow \beta \text{ema}_k + (1 - \beta) \|\Delta_k\|_2$
- 9: $\text{scale}_k \leftarrow c \cdot \|\Delta_k\|_2 \cdot \text{ema}_k$
- 10: **end for**
- 11: $\text{scale}_{\text{total}} \leftarrow \text{SECAGG}(\{\text{scale}_k\}_{k=1}^K)$ and broadcast $\text{scale}_{\text{total}}$
- 12: Each client forms $u_k \leftarrow \theta_r^{(k)} + \text{scale}_{\text{total}} w_k$ and contributes u_k to SECAGG
- 13: Server sets $\theta_r \leftarrow \frac{1}{K} \cdot \text{SECAGG}(\{u_k\}_{k=1}^K)$
- 14: **end for**
- 15: **Verify:** Coalition $|S| \geq t$ computes z using its shares; optionally reconstructs τ to check commitment $(\rho, \mathcal{C})$
- 16: Accept if $z \geq z^*$

clients K (4 to 128) while keeping the global batch size fixed at 2048. We run 300 rounds, each with one local epoch. For CIFAR-10 and CIFAR-100, we hold out 20% of the training data for validation and report test accuracy at the checkpoint with the highest validation accuracy. To evaluate model utility, we report top-1 accuracy. We quantify robustness by measuring the z -score after the attack. We use a detection threshold $z^* = 4$; we vary the watermark strength c as indicated. Additionally, we report a language modeling experiment with GPT-2 Small on WikiText-2. We refer to Appendix C for more implementation details.

Baseline. We compare against a naive per-client watermark

baseline. Each client k samples an independent key τ_k and embeds it locally using the same update rule and scaling hyperparameter c as our method. Verification for client k uses the same one-sided z -test with key τ_k . Under FedAvg aggregation, the watermark direction becomes proportional to $\frac{1}{K} \sum_{k=1}^K \tau_k$, whose expected norm shrinks as $1/\sqrt{K}$, explaining the loss of detectability as K grows.

5.1. Empirical Validation of Normality Assumption

To validate the assumption of normality in the cosine similarity distribution (introduced in Section 4.3), We trained five independent ResNet-18 models for each dataset. For each model θ_i , we computed cosine similarities with 2000 random vectors. We then aggregated the similarities across models to obtain combined distributions from which μ and σ were estimated. As shown in Figure 2, individual model distributions exhibit consistent behavior, and the combined distributions closely match fitted normal curves.

5.2. Watermark Scalability

Figure 3a illustrates the effect of scaling the number of clients on the watermark signal strength for both the baseline and our proposed method. We observe that for the baseline, each client embeds a unique signal, which causes the overall signal to weaken as the number of clients increases. Notably, when $K \geq 16$, the z -score is not statistically significant and falls below our detection threshold for the baseline. In contrast, our collaborative watermarking approach supports scaling to larger K (demonstrated up to $K = 128$) even with a smaller scaling factor ($c = 0.025$, compared to $c = 0.1$ in the baseline). This comparison is conservative for our method. We give the baseline a larger c to improve its detectability at small K , but it still diminishes as K grows. Figure 3b and Table 1 enable matched-utility comparisons by showing how z and accuracy vary with c .

Figure 3b shows that the statistical significance with which our watermark can be detected increases predictably with the watermark strength hyperparameter c (scaling factor). As c increases, the corresponding z -scores rise consistently across all datasets. which shows that our scheme is both

scalable and tunable by adjusting c to control the watermarking strength. In the next section, we further investigate the trade-off between watermark strength and model accuracy.

5.3. Watermark Fidelity

Table 1 shows the model’s accuracy relative to the watermark embedding strength c . At low watermark strengths, the impact on accuracy is small. For example, at $c = 0.025$, test accuracy drops by only 0.12 percentage points (pp) on CIFAR-10, 0.21 pp on CIFAR-100, and 0.25 pp on Tiny ImageNet. Increasing c to 0.05 or 0.075 yields slightly larger but still modest reductions. CIFAR-10 drops by 0.28 pp and 0.64 pp, respectively. CIFAR-100 drops by 1.15 pp and 2.13 pp, and Tiny ImageNet by 0.47 pp and 0.57 pp. This demonstrates that even at moderate strengths, the model maintains high accuracy.

At higher watermark strengths (e.g., $c = 0.1$), accuracy noticeably degrades: 1.1 pp for CIFAR-10, 4.6 pp for CIFAR-100, and 2.5 pp for Tiny ImageNet. Overall, $c = 0.025$ provides a strong default trade-off in our experiments, preserving accuracy while yielding reliable detectability.

Table 1. Test accuracy (%) across datasets for different watermark strengths c ($K = 32$).

c	CIFAR-10	CIFAR-100	TinyImageNet
No Watermark	88.08 $\pm$ 0.53	61.33 $\pm$ 1.56	53.98 $\pm$ 0.88
0.025	87.93 $\pm$ 0.65	61.31 $\pm$ 0.78	53.67 $\pm$ 0.52
0.050	87.62 $\pm$ 0.49	60.47 $\pm$ 0.58	53.51 $\pm$ 0.75
0.075	87.36 $\pm$ 0.13	59.55 $\pm$ 0.98	53.45 $\pm$ 0.68
0.100	86.89 $\pm$ 0.32	58.80 $\pm$ 1.08	52.59 $\pm$ 0.49

5.4. Non-IID Data Heterogeneity

We next evaluate watermark detectability under statistical heterogeneity. We use the setup under a balanced Dirichlet label-skew partition with $K = 32$ and $c = 0.025$. All clients receive the same number of training examples, while class proportions are sampled from a Dirichlet distribution with concentration parameter γ . Smaller γ corresponds to stronger label skew and thus greater statistical heterogeneity.

Table 2 reports the results for $\gamma \in \{0.25, 0.5, 0.75, 1.0\}$. Across all tested values of γ , the watermark remains highly detectable, with z-scores between 12.58 and 12.72, well above the detection threshold $z^* = 4$. The utility gap relative to the non-watermarked model also remains small across all settings. These results suggest that statistical heterogeneity has little effect on the watermark detectability.

Table 2. CIFAR-100 under balanced Dirichlet label skew ($K = 32$).

γ	Z-score	Non-WM Acc.	WM Acc. ($c = 0.025$)
0.25	12.72 $\pm$ 1.61	60.18 $\pm$ 0.55	60.02 $\pm$ 0.28
0.50	12.60 $\pm$ 2.72	61.37 $\pm$ 0.29	61.11 $\pm$ 0.59
0.75	12.58 $\pm$ 2.85	61.16 $\pm$ 0.31	60.86 $\pm$ 0.44
1.00	12.60 $\pm$ 2.88	61.31 $\pm$ 0.11	60.78 $\pm$ 0.07

5.5. Language Modeling

We further evaluate the method beyond image classification tasks. Table 3 summarizes results for GPT-2 Small on WikiText-2. Without watermarking, the model achieves perplexity 10.52 ± 0.05 and z-score 0.17 ± 1.69 , consistent with the absence of a watermark signal. Once watermarking is enabled, detectability remains strong: for $c \in \{0.50, 0.75, 1.00\}$, the z-score stays well above the detection threshold. At the same time, the increase in perplexity is small, indicating that the impact on model utility remains limited in this setting.

Table 3. GPT-2 Small on WikiText-2: perplexity and corresponding z-score ($k = 4$).

c	Perplexity	Z-score
No Watermark	10.52 $\pm$ 0.05	0.17 $\pm$ 1.69
0.50	10.58 $\pm$ 0.03	6.89 $\pm$ 2.51
0.75	10.63 $\pm$ 0.06	8.13 $\pm$ 1.75
1.00	10.70 $\pm$ 0.07	9.63 $\pm$ 1.30

5.6. Watermark Robustness

We evaluate robustness against common post-training removal attacks and adaptive attackers, following prior model-watermarking work (Uchida et al., 2017; Adi et al., 2018; Bansal et al., 2022).

1. **Fine-Tuning:** Fine-tune the released model for 100 epochs using AdamW (1×10^{-3} learning rate, 1×10^{-4} weight decay, batch size 128) on $p\% \in \{1, 5, 10, 20\}$ of the training data.
2. **Quantization:** Weight-only quantization of Conv/Linear layers of the final model checkpoint.
3. **Pruning:** Magnitude pruning (global unstructured) or structured channel pruning with ratios in $\{0.3, 0.5, 0.7, 0.9\}$.
4. **Distillation:** Train a student network for 100 epochs (Adam, 1×10^{-3} , batch size 128) on $p\%$ of the training data with temperature $T = 3$ and loss weight $\alpha = 0.5$.
5. **Adaptive Fine-Tuning:** Use intermediate global checkpoints to estimate a watermark direction and

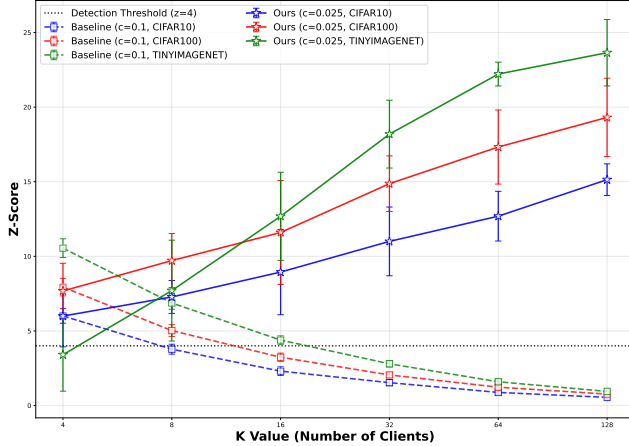
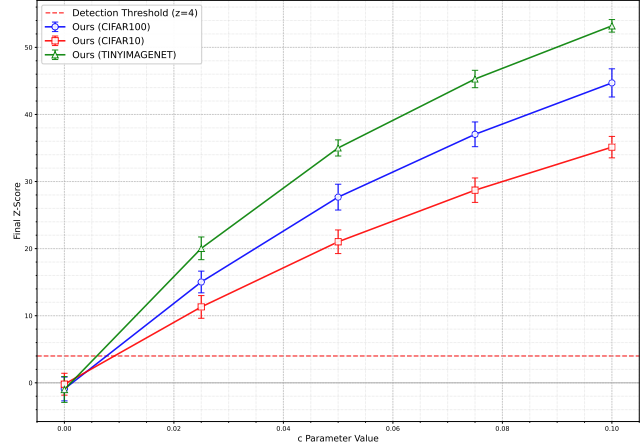

 (a) z -scores vs. number of clients

 (b) z -scores vs. c with $K = 32$.

Figure 3. (a) Our method sustains statistically significant z -scores up to $K = 128$, whereas the baseline collapses beyond $K = 16$. (b) Increasing the scaling factor c consistently boosts z -scores across datasets, showing that watermark strength is tunable.

fine-tune to reduce alignment with the estimate (same data/epoch budgets).

Results. Figure 4 shows the robustness results on CIFAR-100 ($K = 32$, $c = 0.025$) via Pareto frontiers. We observe that even with larger attack budgets (up to 20% of the data) or structural modifications such as 90% pruning, watermark z -scores remain above the detection threshold ($z = 4$). Adaptive fine-tuning yields better results for the attacker, but it still cannot erase the watermark without substantial accuracy degradation. Figure 4 plots all post-attack checkpoints as (accuracy, z) pairs. Points above the dashed line mean that the watermark remained detectable. The dashed Pareto curves summarize the best measured trade-offs at each data budget. With fine-tuning, pruning, and quantization, the watermark remains detectable until the attacker incurs a significant drop in accuracy.

The only attack that reliably removes our watermark is distillation, as expected, since we use a white-box watermark. However, distillation requires (i) a high computational effort to re-train the model and (ii) substantial amounts of training data. We highlight that our watermark is not robust when the adversary has sufficient training data (see the top-left attack in Figure 4). This is well known and agrees with previous works (Boenisch, 2021; Lukas et al., 2021). We refer to Appendix D for more details on robustness.

6. Discussion

We propose threshold watermarking as a problem to be studied, especially as LLMs are being trained using FL (Sani et al., 2024). Our work targets shared model ownership in FL, ensuring that no single participant can unilaterally remove a watermark. The core technical contribution is a pro-

tolocol that combines (i) threshold secret sharing of the watermark key for (t, K) -controlled verification, (ii) distributed watermark embedding under secure aggregation with an untrusted server, and (iii) empirical evaluation of fidelity and robustness at scale. By enforcing a shared embedding direction, our method avoids the detectability loss observed in per-client watermarking baselines as K increases, where individual contributions diminish. Our results show that even with access to the full training trajectory, watermark removal remains difficult under realistic attack budgets. (iv) Experiments under balanced Dirichlet label skew and on GPT-2 Small suggests that the method is not confined to the default IID image-classification setting, where in both cases, watermark detectability remains strong while utility degradation stays limited.

Limitations. Our current instantiation is white-box, meaning that VERIFY assumes access to model parameters. We assume honest-but-curious participants and do not handle training-time adversaries (e.g., Byzantine clients or a malicious server that deviates from the protocol). Extending threshold watermarking to these settings would require additional security and robustness mechanisms. While our experimental evaluation primarily targets image classification, we further assess our method in a GPT-2 Small language model fine-tuning setting. A comprehensive evaluation of larger language models is left for future work. Finally, since the global model trajectory is observable, an attacker may attempt to estimate the watermark key from checkpoints. We empirically instantiate such adaptive removal attacks with substantial auxiliary knowledge, but do not provide a theoretical lower bound on the difficulty of estimating the key; thus, stronger adaptive attacks may exist.

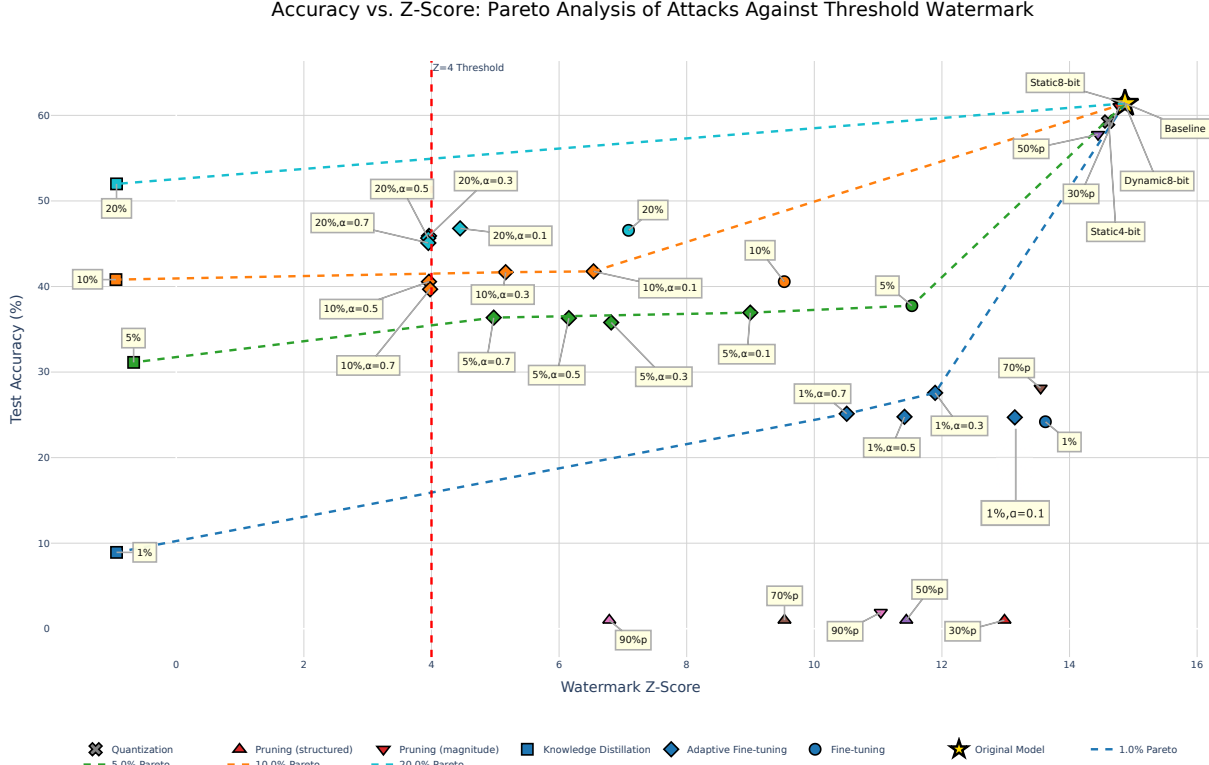


Figure 4. Robustness analysis on CIFAR-100 with $K = 32$ and $c = 0.025$. We report the trade-off between task accuracy and watermark z -score under five attack types: (i) adaptive fine-tuning, (ii) plain fine-tuning, (iii) knowledge distillation, (iv) pruning (magnitude and structured), and (v) quantization. The original model is shown as a star. Dashed curves denote Pareto frontiers for 1%, 5%, 10%, and 20% of the training data, while the red dashed line marks the detection threshold ($z = 4$).

7. Related Work

In centralized training, watermarking embeds a secret signal that later proves ownership of a model (Boenisch, 2021; Lukas et al., 2021). Existing schemes span white-box approaches that encode signals directly in parameters (Lv et al., 2023; Darvish Rouhani et al., 2019; Wang & Kerschbaum, 2021; Wang et al., 2020) and black-box approaches that verify via trigger queries (Adi et al., 2018; Jia et al., 2021; Bansal et al., 2022; Yang et al., 2023b).

In FL, watermarking must account for distributed ownership and limited trust. Secret keys should not be held by a single party, and embedding should remain robust even when the server is untrusted (see Lansari et al. (2023) for a survey). Prior work typically assumes either a trusted server that embeds centrally (Tekgul et al., 2021; Shao et al., 2025; Li et al., 2022) or designated clients that embed via local trigger data (Li et al., 2023; Yang et al., 2023a; Liu et al., 2021; Chen et al., 2024; Liang & Wang, 2023; Xu et al., 2024). Our focus is complementary, since we enable a (t, K) -threshold verification via Shamir secret sharing and enable distributed embedding under secure aggregation.

8. Conclusion

We proposed (t, K) -threshold watermarking for FL, enabling collaborative model ownership without requiring a trusted server. Our construction guarantees that only coalitions of at least t clients can verify provenance, while smaller groups learn nothing beyond the protocol’s output. We instantiated our protocol in the *white-box setting* and empirically demonstrated its robustness against watermark-removal attacks, including adaptive attackers. Our results show that threshold watermarking is practical and scalable to large K , making it suitable for collaborative ownership of machine learning models. More broadly, we hope this work supports large-scale model training and enables reliable attribution through collaborative watermarking.

Impact Statement

This paper introduces a threshold watermarking protocol for federated learning that enables shared ownership and collective provenance verification of jointly trained models. By requiring a coalition of clients to verify a watermark, the method supports collaborative settings in which no single

participant should unilaterally claim ownership or remove attribution. This is particularly relevant for applications such as healthcare or cross-organizational model training, where data and compute are contributed by multiple parties.

References

- Adi, Y., Baum, C., Cisse, M., Pinkas, B., and Keshet, J. Turning your weakness into a strength: watermarking deep neural networks by backdooring. In *Proceedings of the 27th USENIX Conference on Security Symposium, SEC'18*, pp. 1615–1631, USA, 2018. USENIX Association. ISBN 9781931971461.
- Bansal, A., Chiang, P.-Y., Curry, M. J., Jain, R., Wington, C., Manjunatha, V., Dickerson, J. P., and Goldstein, T. Certified neural network watermarks with randomized smoothing. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., and Sabato, S. (eds.), *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 1450–1465. PMLR, 17–23 Jul 2022. URL <https://proceedings.mlr.press/v162/bansal22a.html>.
- Beaufays, F., Rao, K., Mathews, R., and Ramaswamy, S. Federated learning for emoji prediction in a mobile keyboard, 2019. URL <https://arxiv.org/abs/1906.04329>.
- Boenisch, F. A systematic review on model watermarking for neural networks. *Frontiers in Big Data*, Volume 4 - 2021, 2021. ISSN 2624-909X. doi: 10.3389/fdata.2021.729663. URL <https://www.frontiersin.org/journals/big-data/articles/10.3389/fdata.2021.729663>.
- Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, H. B., Patel, S., Ramage, D., Segal, A., and Seth, K. Practical secure aggregation for privacy-preserving machine learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS '17*, pp. 1175–1191, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450349468. doi: 10.1145/3133956.3133982. URL <https://doi.org/10.1145/3133956.3133982>.
- Brassard, G., Chaum, D., and Crépeau, C. Minimum disclosure proofs of knowledge. *Journal of Computer and System Sciences*, 37(2):156–189, 1988.
- Chen, Q., Zheng, P., Du, Y., Luo, W., and Liu, H. Fedmct: A federated framework for intellectual property protection and malicious client tracking. In *Proceedings of the 2024 16th International Conference on Machine Learning and Computing, ICMLC '24*, pp. 80–86, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400709234. doi: 10.1145/3651671.3651710. URL <https://doi.org/10.1145/3651671.3651710>.
- Darvish Rouhani, B., Chen, H., and Koushanfar, F. Deep-signs: An end-to-end watermarking framework for ownership protection of deep neural networks. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '19*, pp. 485–497, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450362405. doi: 10.1145/3297858.3304051. URL <https://doi.org/10.1145/3297858.3304051>.
- Granqvist, F., Seigel, M., van Dalen, R., Áine Cahill, Shum, S., and Paulik, M. Improving on-device speaker verification using federated learning with privacy. In *Interspeech 2020*, pp. 4328–4332, 2020. doi: 10.21437/Interspeech.2020-2944.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016. doi: 10.1109/CVPR.2016.90.
- Jia, H., Choquette-Choo, C. A., Chandrasekaran, V., and Papernot, N. Entangled watermarks as a defense against model extraction. In *30th USENIX Security Symposium (USENIX Security 21)*, pp. 1937–1954. USENIX Association, August 2021. ISBN 978-1-939133-24-3. URL <https://www.usenix.org/conference/usenixsecurity21/presentation/jia>.
- Kairouz, P., McMahan, H. B., Avent, B., Bellet, A., Bennis, M., Bhagoji, A. N., Bonawit, K., Charles, Z., Cormode, G., Cummings, R., D'Oliveira, R. G. L., Eichner, H., El Rouayheb, S., Evans, D., Gardner, J., Garrett, Z., Gascón, A., Ghazi, B., Gibbons, P. B., Gruteser, M., Harchaoui, Z., He, C., He, L., Huo, Z., Hutchinson, B., Hsu, J., Jaggi, M., Javidi, T., Joshi, G., Khodak, M., Konečný, J., Korolova, A., Koushanfar, F., Koyejo, S., Lepoint, T., Liu, Y., Mittal, P., Mohri, M., Nock, R., Özgür, A., Pagh, R., Qi, H., Ramage, D., Raskar, R., Raykova, M., Song, D., Song, W., Stich, S. U., Sun, Z., Theertha Suresh, A., Tramèr, F., Vepakomma, P., Wang, J., Xiong, L., Xu, Z., Yang, Q., Yu, F. X., Yu, H., and Zhao, S. 2021.
- Katz, J. and Lindell, Y. *Introduction to Modern Cryptography*. CRC Press, Boca Raton, FL, USA, 3rd edition, 2020. ISBN 978-0367331756.
- Krizhevsky, A. Learning multiple layers of features from tiny images. Technical Report TR-2009, University of Toronto, 2009.

- Lansari, M., Bellafqira, R., Kapusta, K., Thouvenot, V., Bettan, O., and Coatrieux, G. When federated learning meets watermarking: A comprehensive overview of techniques for intellectual property protection, 2023. URL <https://arxiv.org/abs/2308.03573>.
- Le, Y. and Yang, X. S. Tiny imagenet visual recognition challenge, 2015. Stanford CS231n.
- Li, B., Fan, L., Gu, H., Li, J., and Yang, Q. FedIPR: Ownership Verification for Federated Deep Neural Network Models. *IEEE Transactions on Pattern Analysis & Machine Intelligence*, 45(04):4521–4536, April 2023. ISSN 1939-3539. doi: 10.1109/TPAMI.2022.3195956. URL <https://doi.ieeecomputersociety.org/10.1109/TPAMI.2022.3195956>.
- Li, F.-Q., Wang, S.-L., and Liew, A. W.-C. Watermarking protocol for deep neural network ownership regulation in federated learning. In *2022 IEEE International Conference on Multimedia and Expo Workshops (ICMEW)*, pp. 1–4, 2022. doi: 10.1109/ICMEW56448.2022.9859395.
- Liang, J. and Wang, R. Fedcip: Federated client intellectual property protection with traitor tracking, 2023. URL <https://arxiv.org/abs/2306.01356>.
- Liu, X., Shao, S., Yang, Y., Wu, K., Yang, W., and Fang, H. Secure federated learning model verification: A client-side backdoor triggered watermarking scheme. In *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pp. 2414–2419, 2021. doi: 10.1109/SMC52423.2021.9658998.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- Lukas, N., Jiang, E., Li, X., and Kerschbaum, F. Sok: How robust is image classification deep neural network watermarking? (extended version), 2021. URL <https://arxiv.org/abs/2108.04974>.
- Lv, P., Li, P., Zhang, S., Chen, K., Liang, R., Ma, H., Zhao, Y., and Li, Y. A robustness-assured white-box watermark in neural networks. *IEEE Transactions on Dependable and Secure Computing*, 20(6):5214–5229, 2023. doi: 10.1109/TDSC.2023.3242737.
- Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.
- Muzellec, B., Marteau-Ferey, U., and Marchand, T. Fedpydeseq2: a federated framework for bulk rna-seq differential expression analysis. *bioRxiv*, 2024. doi: 10.1101/2024.12.06.627138. URL <https://www.biorxiv.org/content/early/2024/12/10/2024.12.06.627138>.
- NVIDIA. Federated learning for healthcare using nvidia clara, September 2021. URL <https://developer.download.nvidia.com/CLARA/Federated-Learning-Training-for-Healthcare-Using-NVIDIA-Clara.pdf>. SWE-CLARA-001-USCA.
- Oldenhof, M., Ács, G., Pejő, B., Schuffenhauer, A., Holway, N., Sturm, N., Dieckmann, A., Fortmeier, O., Boniface, E., Mayer, C., Gohier, A., Schmidtke, P., Niwayama, R., Kopecky, D., Mervin, L., Rath, P. C., Friedrich, L., Formanek, A., Antal, P., Rahaman, J., Zalewski, A., Heyndrickx, W., Oluoch, E., Stöbel, M., Vančo, M., Endico, D., Gelus, F., de Boisfossé, T., Darbier, A., Nicolle, A., Blottière, M., Telenczuk, M., Nguyen, V. T., Martinez, T., Boillet, C., Moutet, K., Picosson, A., Gasser, A., Djafar, I., Simon, A., Arany, A., Simm, J., Moreau, Y., Engkvist, O., Ceulemans, H., Marini, C., and Galtier, M. Industry-scale orchestrated federated learning for drug discovery. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI’23/IAAI’23/EAAI’23. AAAI Press, 2023. ISBN 978-1-57735-880-0. doi: 10.1609/aaai.v37i13.26847. URL <https://doi.org/10.1609/aaai.v37i13.26847>.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. Pytorch: an imperative style, high-performance deep learning library. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 2019. Curran Associates Inc.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language models are unsupervised multitask learners. OpenAI Technical Report, 2019.
- Sani, L., Iacob, A., Cao, Z., Marino, B., Gao, Y., Paulik, T., Zhao, W., Shen, W. F., Aleksandrov, P., Qiu, X., and Lane, N. D. The future of large language model pre-training is federated, 2024. URL <https://arxiv.org/abs/2405.10853>.
- Shamir, A. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979. doi: 10.1145/359168.359176.
- Shao, S., Yang, W., Gu, H., Qin, Z., Fan, L., Yang, Q., and Ren, K. FedTracker: Furnishing Ownership Verification and Traceability for Federated Learning Model. *IEEE Transactions on Dependable and Secure Computing*, 22(01):114–131, January 2025. ISSN 1941-0018. doi:

- 10.1109/TDSC.2024.3390761. URL <https://doi.ieeecomputersociety.org/10.1109/TDSC.2024.3390761>.
- Stinson, D. R. *Cryptography: Theory and Practice*. CRC Press, 3rd edition, 2005. ISBN 978-1584885085.
- Tekgul, B. G. A., Xia, Y., Marchal, S., and Asokan, N. Waffle: Watermarking in federated learning. In *2021 40th International Symposium on Reliable Distributed Systems (SRDS)*, pp. 310–320, 2021. doi: 10.1109/SRDS53918.2021.00038.
- Uchida, Y., Nagai, Y., Sakazawa, S., and Satoh, S. Embedding watermarks into deep neural networks. In *Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval, ICMR '17*, pp. 269–277, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450347013. doi: 10.1145/3078971.3078974. URL <https://doi.org/10.1145/3078971.3078974>.
- Wang, J., Wu, H., Zhang, X., and Yao, Y. Watermarking in deep neural networks via error back-propagation. *Electronic Imaging*, 32(4):22–1–22–1, 2020. doi: 10.2352/ISSN.2470-1173.2020.4.MWSF-022. URL <https://library.imaging.org/ei/articles/32/4/art00003>.
- Wang, T. and Kerschbaum, F. Riga: Covert and robust white-box watermarking of deep neural networks. In *Proceedings of the Web Conference 2021, WWW '21*, pp. 993–1004, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383127. doi: 10.1145/3442381.3450000. URL <https://doi.org/10.1145/3442381.3450000>.
- Xu, Y., Tan, Y., Zhang, C., Chi, K., Sun, P., Yang, W., Ren, J., Jiang, H., and Zhang, Y. Robwe: Robust watermark embedding for personalized federated learning model ownership protection, 2024. URL <https://arxiv.org/abs/2402.19054>.
- Yang, T., Andrew, G., Eichner, H., Sun, H., Li, W., Kong, N., Ramage, D., and Beaufays, F. Applied federated learning: Improving google keyboard query suggestions, 2018. URL <https://arxiv.org/abs/1812.02903>.
- Yang, W., Shao, S., Yang, Y., Liu, X., Liu, X., Xia, Z., Schaefer, G., and Fang, H. Watermarking in secure federated learning: A verification framework based on client-side backdooring. *ACM Trans. Intell. Syst. Technol.*, 15(1), December 2023a. ISSN 2157-6904. doi: 10.1145/3630636. URL <https://doi.org/10.1145/3630636>.
- Yang, W., Yin, Y., Zhu, G., Gu, H., Fan, L., Cao, X., and Yang, Q. Fedzpk: Federated model ownership verification with zero-knowledge proof, 2023b. URL <https://arxiv.org/abs/2305.04507>.
- Ye, R., Wang, W., Chai, J., Li, D., Li, Z., Xu, Y., Du, Y., Wang, Y., and Chen, S. Openfedllm: Training large language models on decentralized private data via federated learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD '24*, pp. 6137–6147, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400704901. doi: 10.1145/3637528.3671582. URL <https://doi.org/10.1145/3637528.3671582>.

A. Notation Table

Table 4. Notation table for collaborative threshold watermarking.

Symbol	Definition
K	Total number of clients in federated learning (FL).
k	Index of a client, $k \in \{1, \dots, K\}$.
D_k	Local dataset of client k .
n_k	Number of samples in D_k .
n	Total number of samples, $n = \sum_{k=1}^K n_k$.
$F_k(\theta)$	Local loss function of client k .
$F(\theta)$	Global objective, weighted sum of local losses.
θ_r	Global model parameters at round r .
θ_0	Initial global model.
θ_w	Final watermarked global model after T rounds.
$\theta_r^{(k)}$	Locally trained model of client k at round r .
$\nabla \theta_r^{(k)}$	Local model update of client k at round r .
θ_s	Suspect model under verification.
τ	Secret watermarking key vector.
τ'	Estimated key (attack approximation of τ).
$\mathcal{C}$	Public commitment to τ (from dealer).
ρ	Public nonce (randomness) used in the commitment $\mathcal{C}$
s_i	Shamir secret share of τ held by client i .
w_i	Additive share of τ (trustless-server setting).
S_r	Set of active clients in round r (participating clients).
$a_{k,r}$	Normalized aggregation weight of client $k \in S_r$ in round r , with $\sum_{k \in S_r} a_{k,r} = 1$.
$\tilde{w}_{k,r}$	Weighted embedding share of client k in round r , defined as $\tilde{w}_{k,r} = a_{k,r} w_k$.
c	Scaling constant for watermark strength.
β	EMA decay factor for update magnitudes.
ema_k	Exponential moving average tracker for client k .
scale_k	Scaling factor for client k 's watermark perturbation.
$\text{scale}_{\text{total}}$	Aggregated watermark scaling factor.
z	Standardized verification statistic.
z^*	Detection threshold for watermark verification.
μ, σ	Mean and standard deviation of cosine similarities (unwatermarked models).
E	Local training epochs per round.
T	Number of global FL rounds.
η	Learning rate.
z_s, z_t	Student and teacher logits (distillation attack).
T_{distill}	Temperature parameter for distillation softening.
α	Mixing factor (attack or distillation trade-off).
$p\%$	Fraction of training data available to adversary during attacks.

B. Dealer-Free Setup (DKG)

If a trusted dealer is unavailable, clients can generate shares of a random watermark key without any single party learning τ in the clear using a standard distributed key generation (DKG) protocol. At a high level, each client contributes randomness, and the resulting secret key is implicitly defined as the sum of all contributions.

B.1. Dealer-Free Setup Protocol

Algorithm 3 describes a dealer-free setup based on Shamir secret sharing. Each client k samples an additive share w_k and constructs a degree- $(t - 1)$ polynomial P_k with constant term $P_k(0) = w_k$. Clients privately exchange polynomial evaluations and locally aggregate them to obtain Shamir shares s_i of the implicit secret $\tau = \sum_{k=1}^K w_k$. No single client ever reconstructs τ , and Shamir’s (t, K) -threshold secrecy is preserved.

Algorithm 3 Dealer-free setup via distributed key generation (DKG).

Require: K clients, threshold t

```

1: Publicly sample nonce  $\rho$ 
2: for each client  $k$  in parallel do
3:   Sample additive share  $w_k \sim \mathcal{N}(0, I_d/K)$ 
4:   Sample degree- $(t - 1)$  polynomial  $P_k$  with  $P_k(0) = w_k$ 
5:   for each client  $i \in \{1, \dots, K\}$  do
6:     Privately send  $P_k(i)$  to client  $i$ 
7:   end for
8: end for
9: for each client  $i$  do
10:  Set Shamir share  $s_i \leftarrow \sum_{k=1}^K P_k(i)$ 
11:  Keep additive share  $w_i$ 
12: end for
```

B.2. Dealer-Free Setup Overhead

Figure 5 reports the per-client runtime of the DKG protocol as a function of the number of participating clients K , measured under a simulated 1000 Mbps network. We decompose the total runtime into computation and communication costs. The observed scaling matches the theoretical complexity: for fixed model dimension d , per-client communication grows as $O(Kd)$ and per-client computation as $O(Ktd)$, consistent with Shamir-based DKG. At the system level, this corresponds to an overall communication complexity of $O(K^2d)$ and a total computation complexity of $O(K^2td)$. As K increases, the threshold t grows proportionally, leading to computation dominating communication, which explains the widening gap at larger scales.

C. Implementation Details

Datasets. We conduct experiments on CIFAR-10, CIFAR-100 (Krizhevsky, 2009), and Tiny ImageNet (Le & Yang, 2015). Since CIFAR-10 and CIFAR-100 do not provide validation sets, we split the original training set into 80% for training and 20% for validation. For evaluation, we select the model checkpoint with the highest validation accuracy. For CIFAR-10 and CIFAR-100, we apply the AutoAugment policy designed for CIFAR-10. For Tiny ImageNet, training images are augmented using random horizontal flips ($p = 0.5$), random rotations ($\pm 15^\circ$), and color jittering (brightness, contrast, and saturation up to 0.4; hue up to 0.1). For the non-IID CIFAR-100 experiment, we use a balanced Dirichlet label-skew partition in which all clients receive the same number of training examples, while class proportions are sampled from a Dirichlet distribution with concentration parameter $\gamma \in \{0.25, 0.5, 0.75, 1.0\}$.

Model. All experiments use the ResNet-18 architecture (He et al., 2016), with models initialized randomly before training. Implementations and training are carried out in PyTorch (Paszke et al., 2019).

Training. In our FL setup, we vary the number of clients from 4 to 128 while keeping the global batch size fixed at 2048, ensuring an equal number of batches across experiments. We use the AdamW optimizer (Loshchilov & Hutter, 2019) with a learning rate of 1×10^{-3} , weight decay of 1×10^{-4} , and betas (0.9, 0.999). Training runs for 300 rounds with a single epoch per round. The EMA decay factor β for our watermark is set to 0.9. To ensure reproducibility and reduce the risk of seed overfitting, all experiments are repeated with three random seeds: 0, 1, and 2.

Language modeling. We additionally evaluate GPT-2 Small on WikiText-2 in a federated fine-tuning setting. We use $K = 4$ clients, train for 20 federated rounds with AdamW, and use a learning rate of 1×10^{-4} . For evaluation, we retain the checkpoint with the lowest perplexity.

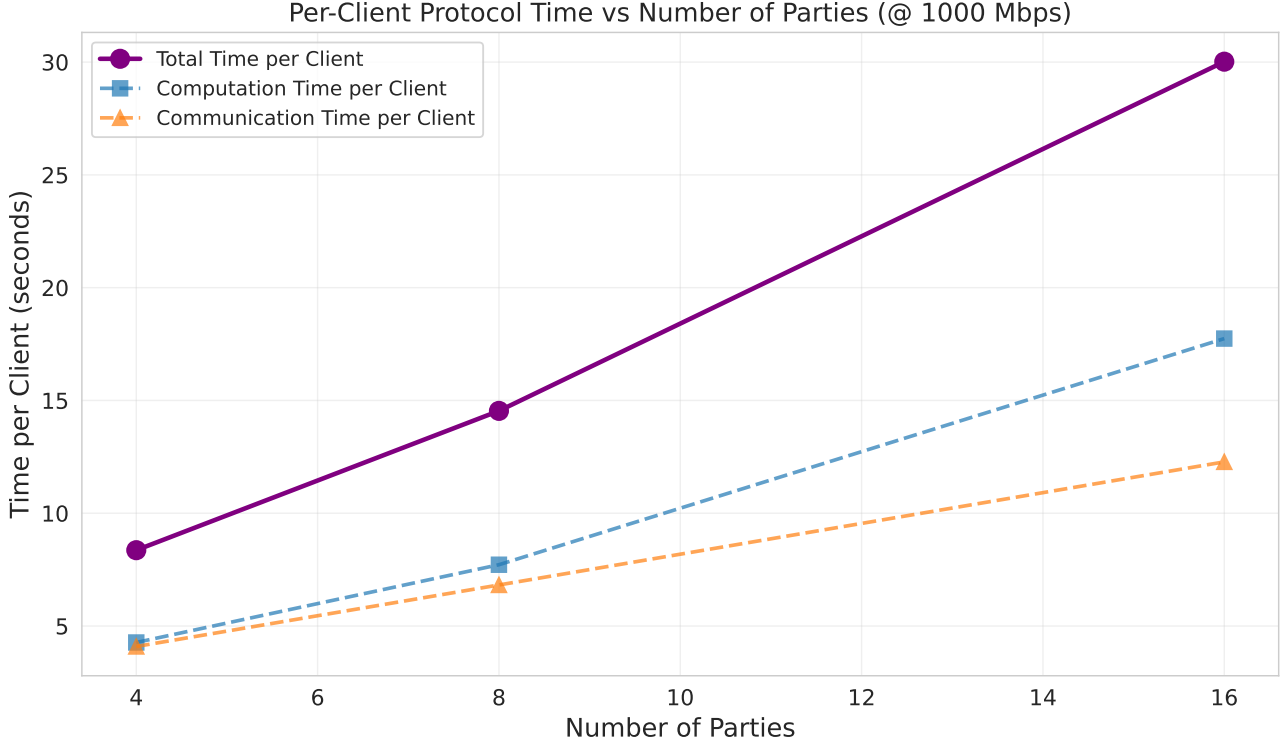


Figure 5. Per-client DKG protocol runtime versus number of participating clients under a 1000 Mbps network, showing total time and the breakdown between computation and communication.

Hardware. All training and post-training attack experiments are conducted on a single NVIDIA RTX A6000 GPU (48 GB VRAM).

D. Robustness Analysis

This section details the robustness evaluation of our watermarking method. We describe the attacks used to showcase our watermark robustness, along with the experimental setup, so that our results can be reproduced and extended by other researchers. Robustness is assessed under a range of attack scenarios, with measurements reported in terms of (i) classification accuracy on the test set and (ii) watermark detectability via the standardized z -score.

D.1. Fine-Tuning

Fine-tuning is the most direct strategy an attacker can attempt, simply retrain the released model on a subset of data in the hope of diminishing the watermark signal.

The attack is assumed to have access to only a subset of the training data, with size $p\% \in \{1, 5, 10, 20\}$ of the full training dataset. Each subset is sampled uniformly at random from the training set with fixed seeds to ensure reproducibility. Fine-tuning proceeds for 100 epochs using the AdamW optimizer with learning rate 1×10^{-3} , weight decay 1×10^{-4} , betas (0.9, 0.999), and batch size 128.

Fine-tuning gradually reduces the alignment between model parameters and watermark flip vectors, but does not cross the decision threshold within 100 epochs; on the other hand, the model utility is diminished by a very high margin even with 20% of the available data, as shown in Figure 6.

D.2. Adaptive Fine-Tuning

We assume an adaptive attacker who participated during training and estimate the watermark direction by accumulating gradient information from intermediate checkpoints. Let τ' denote the resulting estimated key, and let θ denote the current

model parameters during fine-tuning. The attacker then fine-tunes the model to reduce its alignment with τ' by minimizing

$$\mathcal{L}'(\theta) = (1 - \alpha)\mathcal{L}(\theta) + \alpha |\cos(\theta, \tau')| \quad (12)$$

where $\mathcal{L}$ is the model’s utility loss, and α controls the tradeoff between the utility loss function and the removal objective. When $\alpha = 0$, this reduces to standard fine-tuning. We evaluate $\alpha \in \{0.1, 0.3, 0.5, 0.7\}$. Other hyperparameters (epochs, optimizer, batch size) match fine-tuning. For each epoch, we log accuracy and z -score. Pareto frontiers are constructed by retaining non-dominated points in the (accuracy, z) plane.

Optimizing to minimize the signal of the estimated key improves the effectiveness of the fine-tuning attack. Increasing α drives z down more rapidly, but once z falls below the detection threshold ($z < 4$), the model also exhibits very high accuracy degradation.

D.3. Quantization

Quantization discretizes model weights to a lower numerical precision. Quantization threatens embedded signals because fine-grained correlations in parameter space may be destroyed by discretization.

We apply symmetric weight-only quantization to all Conv/Linear layers, leaving biases and BatchNorm parameters unchanged. Three settings are considered:

- **Static8:** per-tensor symmetric 8-bit quantization,
- **Static4:** per-tensor symmetric 4-bit quantization,
- **Dynamic8:** per-output-channel symmetric 8-bit quantization.

For per-tensor quantization, a single scale is computed as the maximum absolute value in the tensor divided by the representable integer range. For per-channel quantization, scales are computed independently for each output channel.

Results shown in Figure 8 indicate that all quantized models still carry the watermark signal, regardless of the quantization method used.

D.4. Pruning

Pruning reduces model size by eliminating parameters. Both unstructured (sparsity-inducing) and structured (channel-removal) pruning can disrupt embedded watermarks by discarding weights that carry watermark information. We evaluate two pruning strategies:

- **Magnitude pruning:** global unstructured pruning across all Conv/Linear weights, removing a fraction with the smallest magnitude.
- **Structured pruning:** per-layer pruning of entire output channels, removing a fraction of channels per module.

Pruning ratios $\in \{0.3, 0.5, 0.7, 0.9\}$ are tested. Models are evaluated immediately after pruning without retraining. Structured pruning is implemented with ℓ_1 -norm channel selection.

Both pruning strategies reduce the watermark signal as the pruning ratio increases, but never cross the decision threshold as shown in Figure 9.

D.5. Knowledge Distillation

An attacker can attempt to distill the model by training a student network to imitate the watermarked teacher.

The student is trained using a mixture of ground-truth supervision and distillation from teacher logits. Given temperature $T = 3.0$ and weighting $\alpha = 0.5$, the loss is

$$\mathcal{L} = \alpha \text{KL}\left(\sigma\left(\frac{z_s}{T}\right) \parallel \sigma\left(\frac{z_t}{T}\right)\right) + (1 - \alpha) \text{CE}(z_s, y) \quad (13)$$

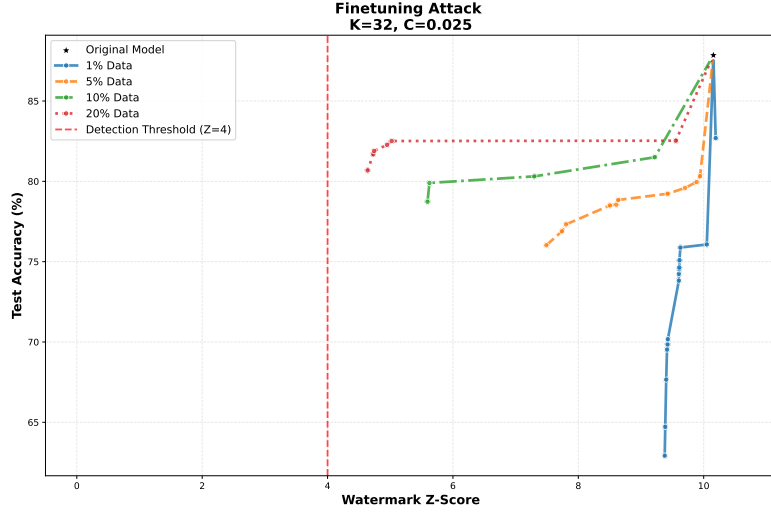
where z_s and z_t denote the student and teacher logits.

The student is trained for 100 epochs with Adam (learning rate 1×10^{-3} , batch size 128) on subsets $p\% \in \{1, 5, 10, 20\}$ of the exported training data. We report test accuracy, transfer rate, and final watermark z -score.

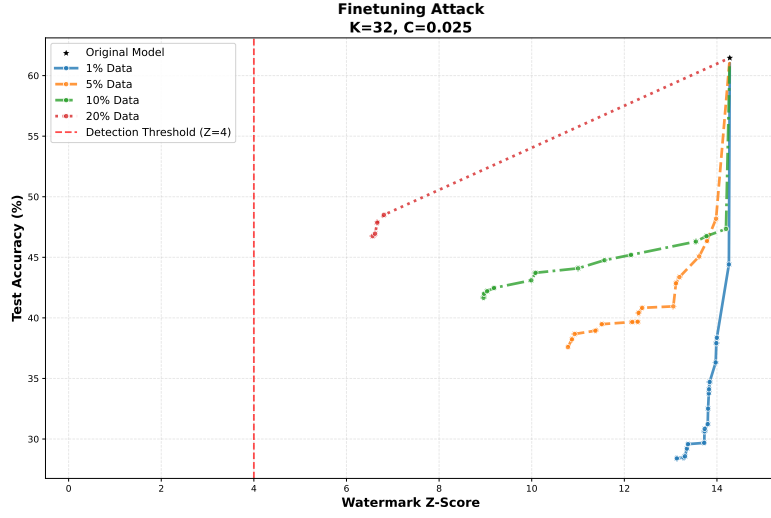
Distillation can produce student models that approach the teacher’s task accuracy while noticeably attenuating the watermark signal, since the functional behavior is transferred without preserving parameter-level correlations. However, we observe that distillation doesn’t produce a high utility model when the available data fraction is below 20%, illustrated in Figure 10, meaning that a substantial portion of the training data is required for the attack to succeed. Moreover, training a new student network is computationally expensive, making this approach significantly more costly than lightweight post-processing attacks such as pruning or quantization.

D.6. Pareto graphs

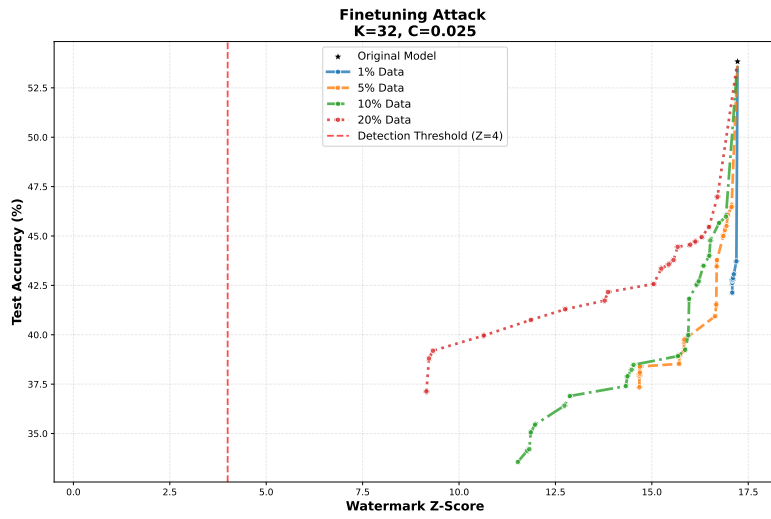
We extend the accuracy– z -score Pareto analysis to CIFAR-10 and Tiny ImageNet. Across both datasets, the watermark exhibits the same robustness patterns observed in the main CIFAR-100 results: post-training attacks must induce a clear degradation in task accuracy before reducing the watermark statistic below the detection threshold ($z \geq 4$). In particular, quantization has negligible impact on detectability, while fine-tuning and pruning trade accuracy for only gradual reductions in z -score. These results confirm that the observed robustness behavior is consistent across datasets of varying difficulty and scale.



(a) CIFAR-10

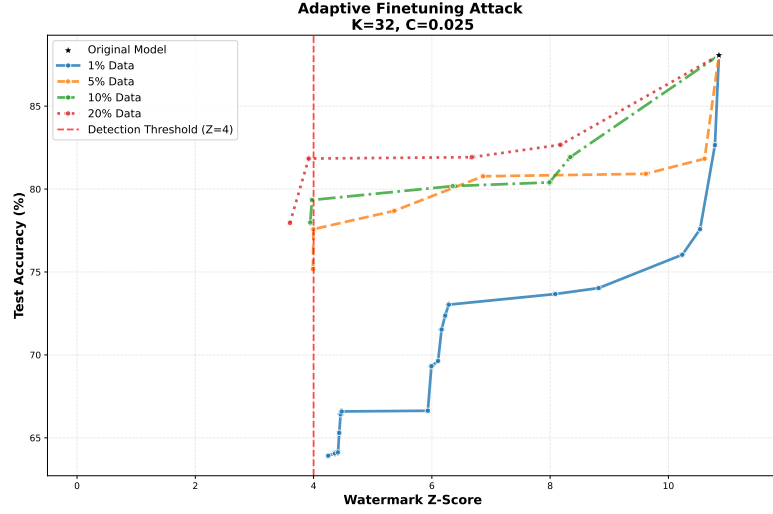


(b) CIFAR-100

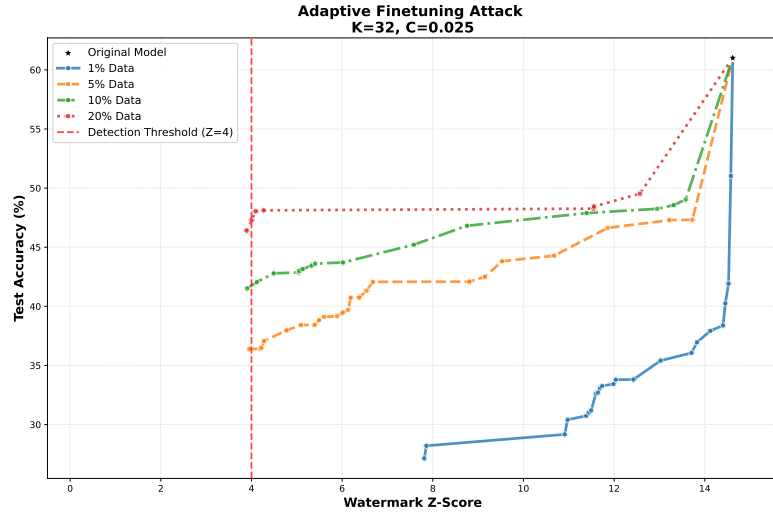


(c) TinyImageNet

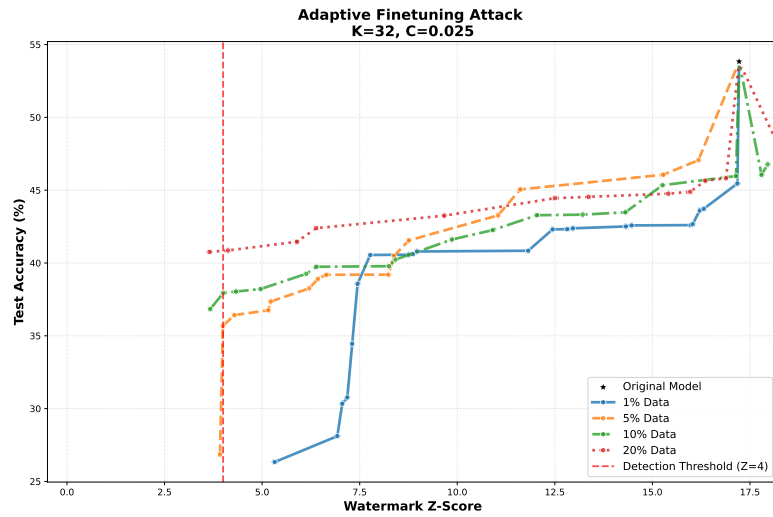
Figure 6. Fine-tuning attack: test accuracy versus watermark z -score under different fractions of fine-tuning data (5%, 10%, 20%). The dashed red line denotes the detection threshold ($z = 4.0$).



(a) CIFAR-10

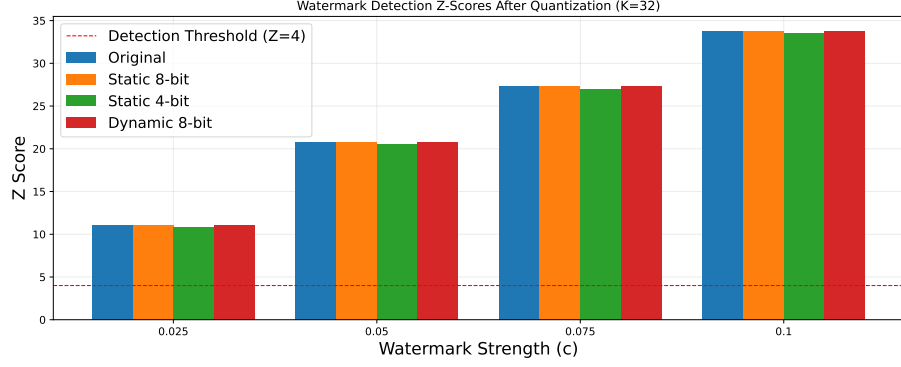


(b) CIFAR-100

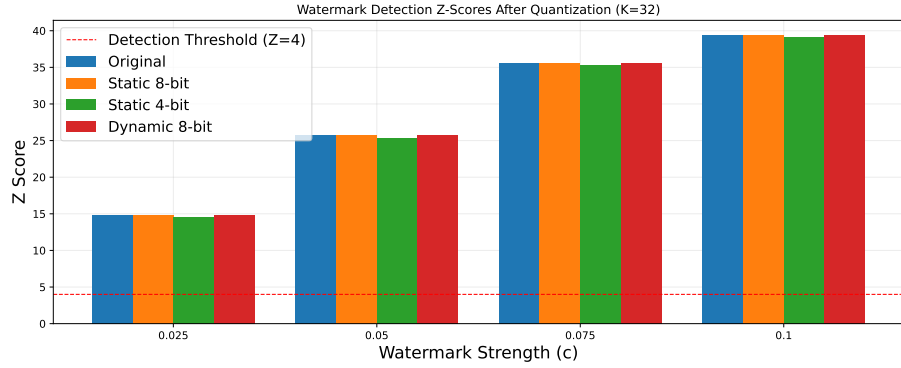


(c) TinyImageNet

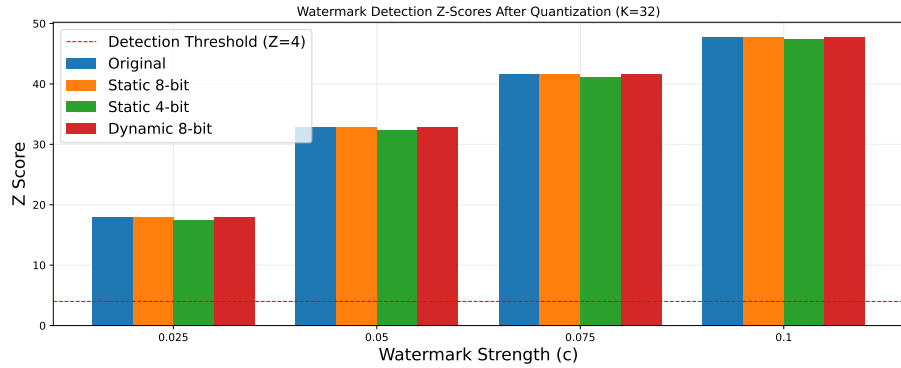
Figure 7. Adaptive fine-tuning attacks with $K = 32$ clients and varying watermark strengths ($c \in \{0.075, 0.05, 0.025\}$). Each curve shows the trade-off between test accuracy and watermark detectability (z -score) as the adversary fine-tunes the watermarked model with different fractions of training data (1%, 5%, 10%, 20%). The red dashed line marks the detection threshold ($z^* = 4$).



(a) CIFAR-10



(b) CIFAR-100



(c) TinyImageNet

Figure 8. Watermark detection z -scores after quantization. Results are shown for CIFAR-10 (a), CIFAR-100 (b), and TinyImageNet (c) under varying watermark strengths c . Each panel compares the original model with three quantization schemes. The dashed red line indicates the detection threshold ($z = 4$).

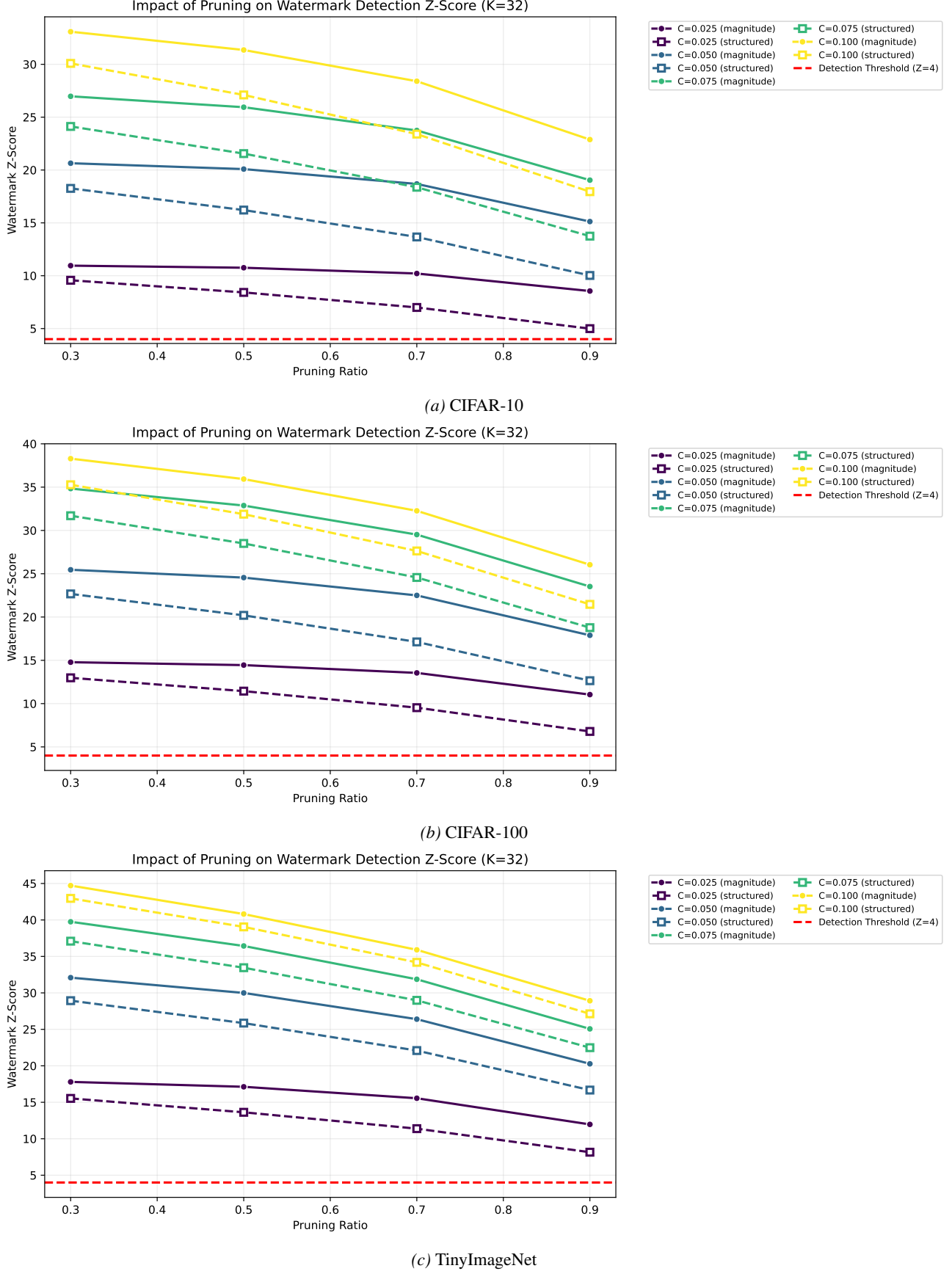
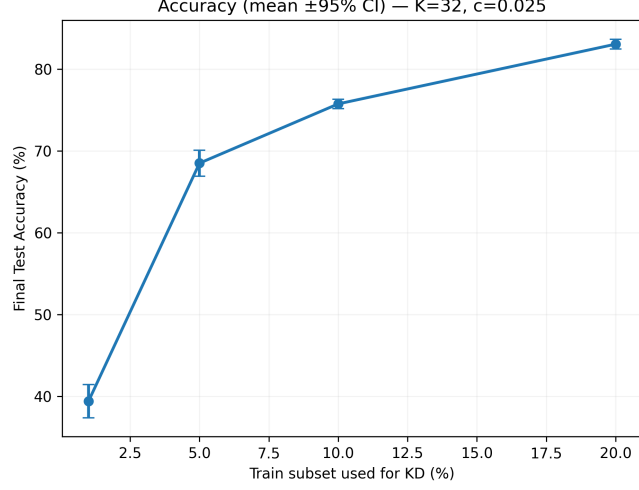
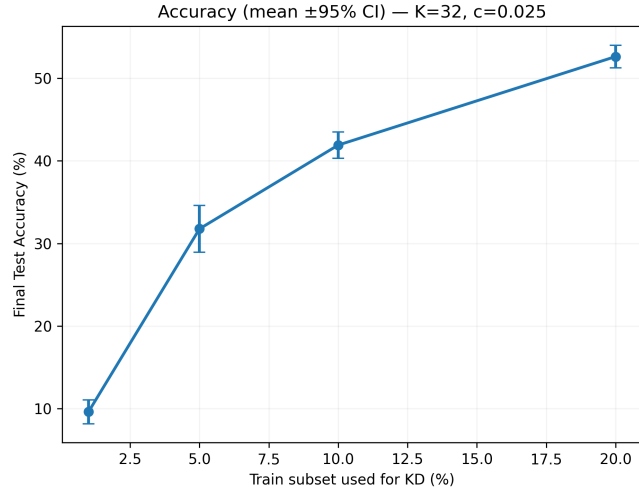


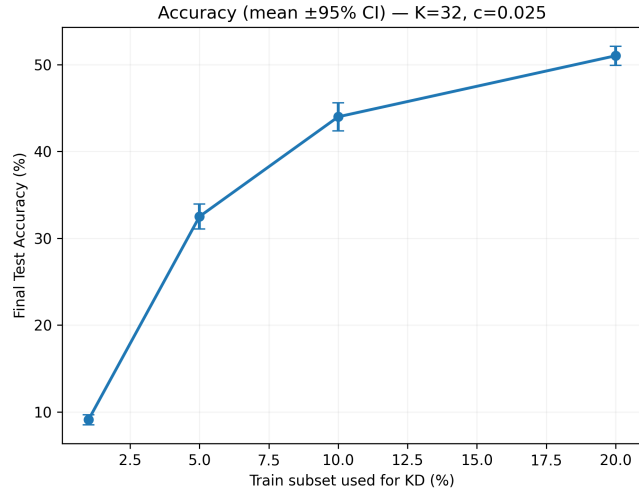
Figure 9. Impact of pruning on watermark detection z -scores across datasets. Results are shown for (a) CIFAR-10, (b) CIFAR-100, and (c) TinyImageNet.



(a) CIFAR-10



(b) CIFAR-100



(c) TinyImageNet

Figure 10. Distillation attack with $K = 32$ clients and watermark strength $c = 0.025$. Final test accuracy is shown as the fraction of training data used for knowledge distillation increases. Results are reported for (a) CIFAR-10, (b) CIFAR-100, and (c) TinyImageNet. Error bars denote 95% confidence intervals across seeds.

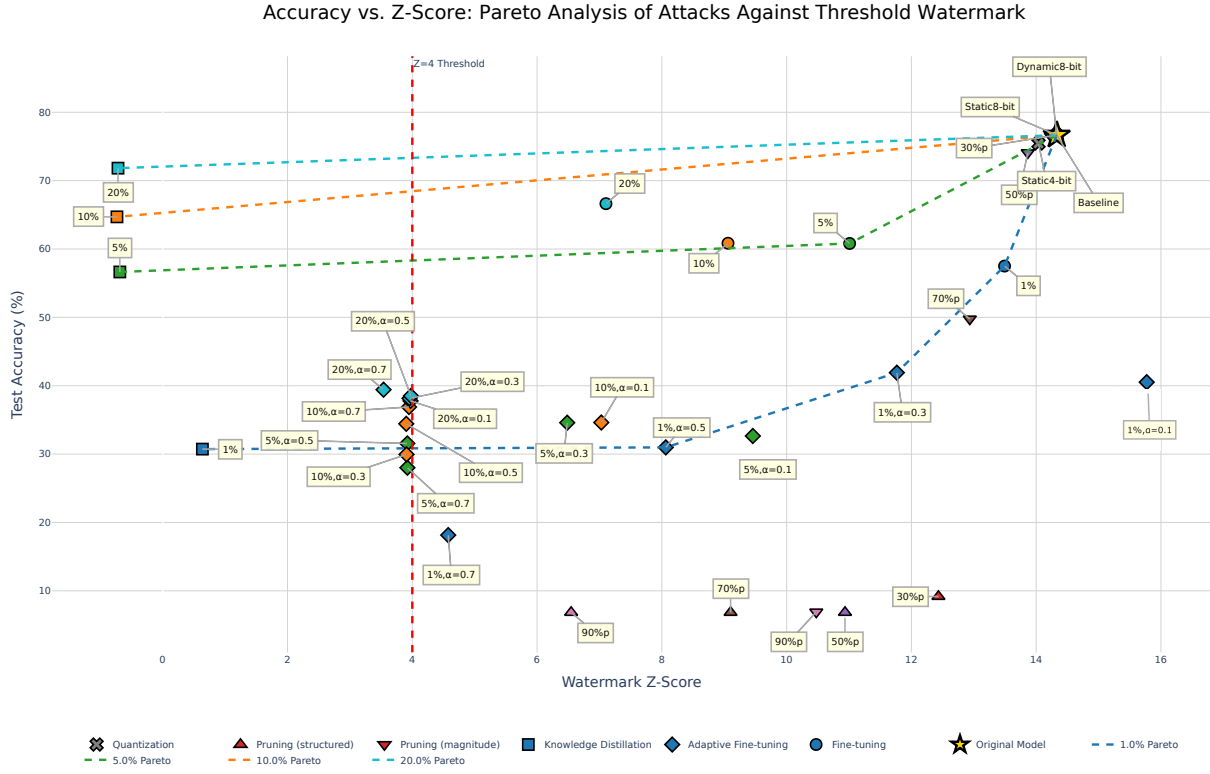


Figure 11. Robustness analysis on Tinyimagenet with $K = 32$ and $c = 0.025$. We report the trade-off between task accuracy and watermark z -score under five attack types: (i) adaptive fine-tuning, (ii) plain fine-tuning, (iii) knowledge distillation, (iv) pruning (magnitude and structured), and (v) quantization. The original model is shown as a star. Dashed curves denote Pareto frontiers for 1%, 5%, 10%, and 20% of the training data, while the red dashed line marks the detection threshold ($z = 4$).

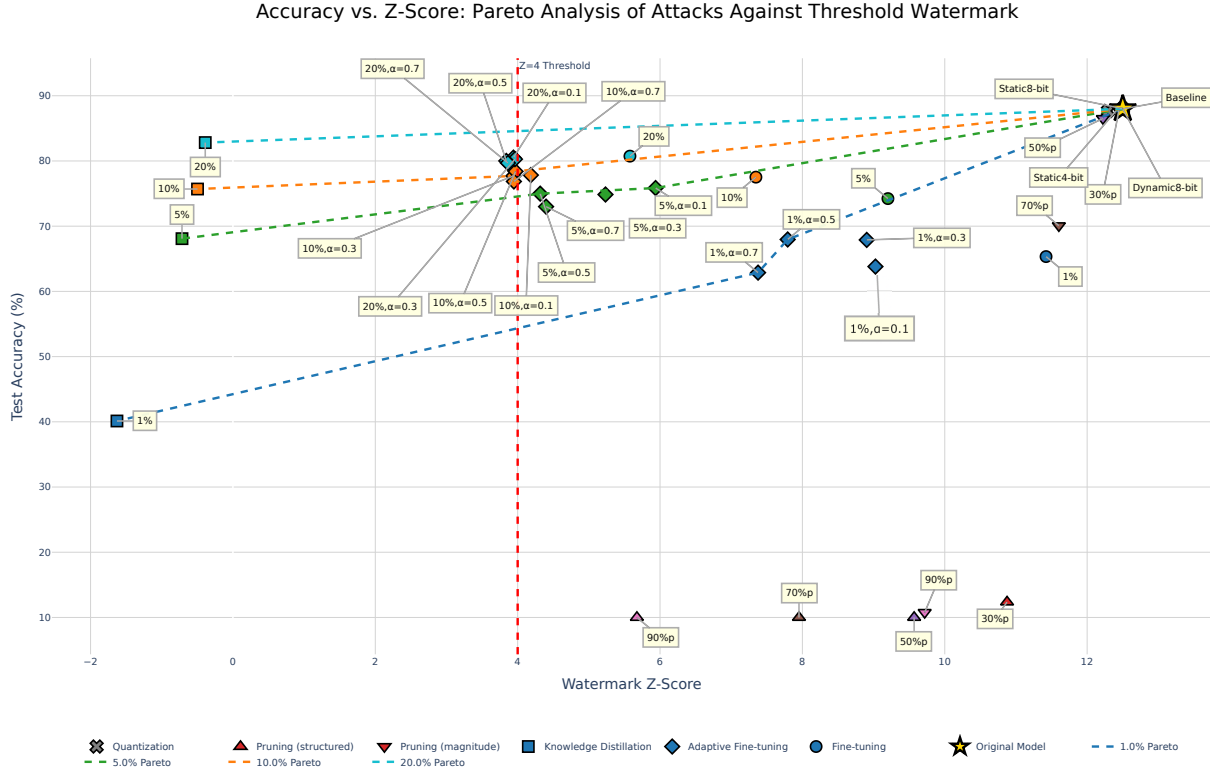


Figure 12. Robustness analysis on CIFAR-10 with $K = 32$ and $c = 0.025$. We report the trade-off between task accuracy and watermark z -score under five attack types: (i) adaptive fine-tuning, (ii) plain fine-tuning, (iii) knowledge distillation, (iv) pruning (magnitude and structured), and (v) quantization. The original model is shown as a star. Dashed curves denote Pareto frontiers for 1%, 5%, 10%, and 20% of the training data, while the red dashed line marks the detection threshold ($z = 4$).